Lab 2: 2-regret heuristics

Authors: Piotr Foltyniewicz 160295, Jakub Adamski 160291

Problem description

Each instance consists of 200 nodes. A row in the input file contains the node's planar coordinates and selection cost, in the form `x;y;cost`. Node indices are zero-based and correspond to row order. The task is to select exactly 100 distinct nodes (half of the available nodes), order them, and form a closed cycle that minimizes the sum of travel distances and selected-node costs.

Distances are Euclidean, rounded to the nearest integer and precomputed after reading the instance. All algorithms use this distance matrix. The evaluation sums all selected-node costs and all cycle edges, including last -> first. In the pseudocode, `m = 100`, `c[v]` is node cost, and `d(a, b)` is the precomputed distance.

The objective for an ordered selection `S` is:

```
F(S) = sum(c[S[i]]) + sum(d(S[i], S[(i + 1) mod length(S)]))
```

Pseudocode

All indices are zero-based. The four methods combine two insertion models (open path for nearest neighbor, closed cycle for greedy cycle) with two node-selection criteria (2-regret and weighted sum). Every method starts from one node and inserts unused nodes until `m = 100`. The final objective always includes the closing edge, also for the open-path variants.

Insertion costs

For nearest neighbor, positions `0, ..., length(S)` include both endpoints and all internal positions. The closing edge is excluded from construction decisions. Greedy cycle considers positions `1, ..., length(S)`, including insertion into the last-to-first edge.

```
INSERTION_COST(S, v, position, cycle):
    left = S[position - 1] if position > 0 else NONE
    right = S[position] if position < length(S) else NONE
    if cycle and right == NONE:
        right = S[0]
    delta = c[v]
    if left != NONE:
        delta += d(left, v)
    if right != NONE:
        delta += d(v, right)
    if left != NONE and right != NONE:
        delta -= d(left, right)
    return delta
```

2-regret calculation

For each unused node, the two cheapest insertion positions are found. Regret is the difference between their costs. Two distinct positions with equal costs give regret zero. If only one position exists (the initial greedy-cycle step), regret is zero as well; its insertion cost is `2 * d(start, v) + c[v]`.

```
CALCULATE_REGRET(S, v, cycle):
    best = infinity
    secondBest = infinity
    bestPosition = NONE
    positions = 1, ..., length(S) if cycle else 0, ..., length(S)
    for position in positions in increasing order:
        delta = INSERTION_COST(S, v, position, cycle)
        if delta < best:
            secondBest = best
            best = delta
            bestPosition = position
        else if delta < secondBest:
            secondBest = delta
    regret = 0 if secondBest == infinity else secondBest - best
    return regret, best, bestPosition
```

Nearest neighbor and greedy cycle: 2-regret

The unused node with the largest regret is selected and inserted at its cheapest position. Equal regrets are resolved by smaller insertion cost, then smaller node index. Equal insertion costs for one node keep the first position.

```
REGRET_CONSTRUCTION(start, cycle):
    S = [start]
    U = V minus {start}
    while length(S) < m:
        for v in U in increasing index order:
            regret[v], best[v], position[v] = CALCULATE_REGRET(S, v, cycle)
        v = node maximizing regret[v], then minimizing best[v], then index
        insert v into S at position[v]
        remove v from U
    return S, evaluateClosedCycle(S)
```

Nearest neighbor and greedy cycle: weighted sum

The weighted criterion balances low insertion cost with high regret. The implementation uses `w = 0.5`, minimizing `(1 - w) * best - w * regret`, equivalently `best - regret`. Node cost is included in each insertion cost; it cancels in the regret difference but remains part of the weighted criterion. Equal scores keep the first node in increasing index order.

```
WEIGHTED_CONSTRUCTION(start, cycle):
    S = [start]
    U = V minus {start}
    w = 0.5
    while length(S) < m:
        for v in U in increasing index order:
            regret[v], best[v], position[v] = CALCULATE_REGRET(S, v, cycle)
            score[v] = (1 - w) * best[v] - w * regret[v]
        v = node minimizing score[v], then index
        insert v into S at position[v]
        remove v from U
    return S, evaluateClosedCycle(S)
```

All variants recompute insertion costs at each step without a cache.

Results

Each method was tested 200 times on TSPA and TSPB, starting once from each node. Lower objective values are better. Tables use the current exported summaries; times are averages per run.

Instance	Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)
TSPA	Greedy Cycle 2-regret heuristics	200	112806	128089	120378.76	5.229371
TSPA	Greedy Cycle heuristics with a weighted sum criterion	200	72011	75532	73353.51	5.075577
TSPA	NN heuristics 2-regret heuristics	200	110950	135771	121360.29	5.459556
TSPA	NN heuristics with a weighted sum criterion	200	72419	78504	75561.20	5.377398
TSPB	Greedy Cycle 2-regret heuristics	200	68150	77274	72484.84	5.307665
TSPB	Greedy Cycle heuristics with a weighted sum criterion	200	52731	63370	57866.33	5.140883
TSPB	NN heuristics 2-regret heuristics	200	68761	79756	73649.84	5.374936
TSPB	NN heuristics with a weighted sum criterion	200	50485	61406	55698.34	5.332107

Lab 1 results for comparison

Instance	Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)
TSPA	Random	200	233146	288073	260752.93	0.007843
TSPA	Nearest Node End	200	84362	90075	87476.63	0.060209
TSPA	Nearest Node Any	200	72328	77022	74306.53	2.884855
TSPA	Greedy Cycle	200	72590	75281	73743.73	3.582711
TSPB	Random	200	179244	236859	206352.15	0.010232
TSPB	Nearest Node End	200	56007	63166	59397.33	0.086600
TSPB	Nearest Node Any	200	52915	66164	59476.22	4.494223
TSPB	Greedy Cycle	200	55655	63734	59650.81	3.416733

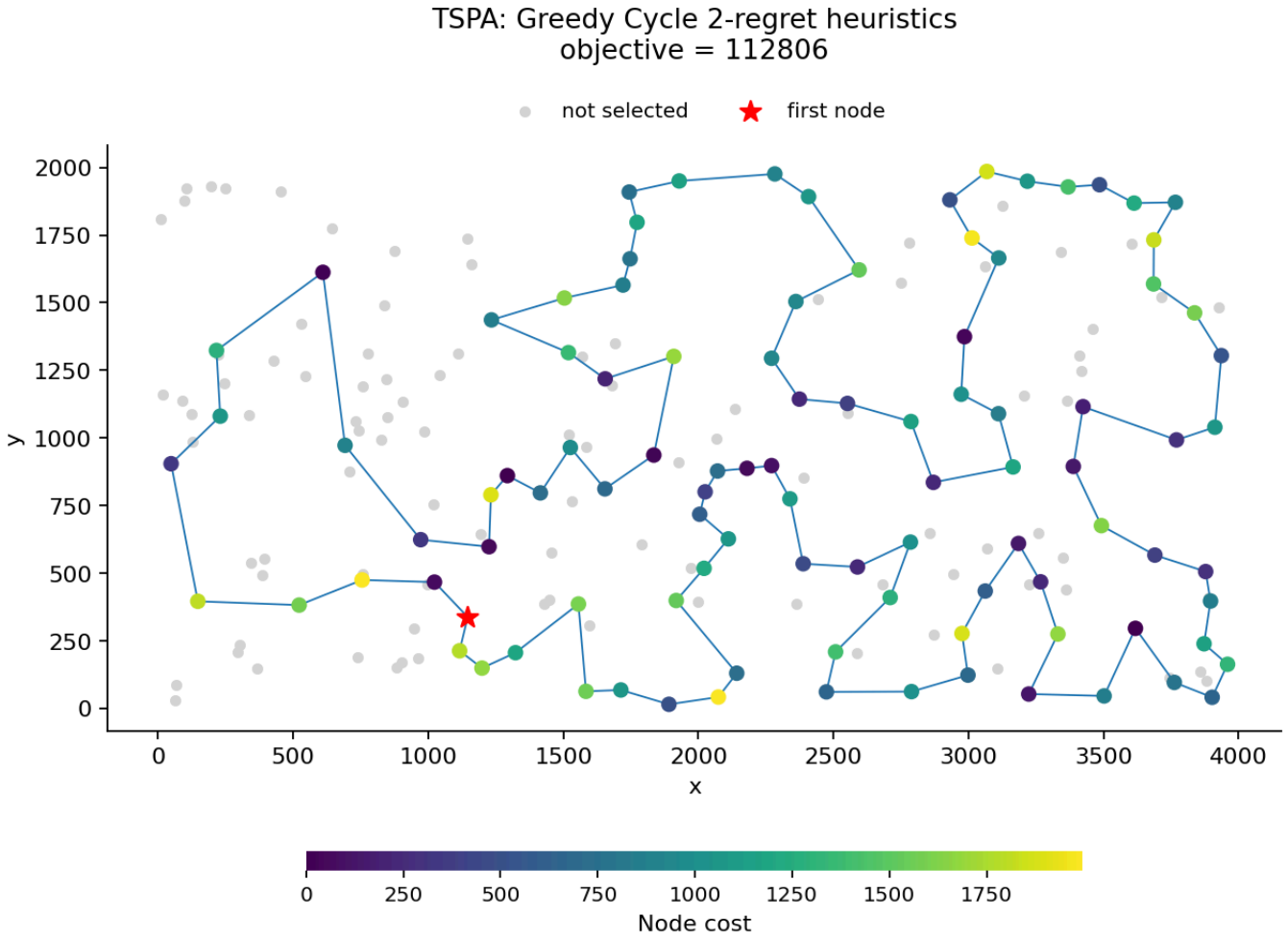
Best solutions and visualizations

Node color represents selection cost; the compact horizontal scale below each plot uses the same cost range for all methods on an instance. Gray nodes are not selected, blue segments form the closed tour, and the red star marks the first node in the displayed sequence. This may differ from the construction start when insertion before the first node is allowed. Lists below show ten node indices per row; read left to right, then top to bottom to follow the tour.

TSPA

Greedy Cycle 2-regret heuristics

Objective: **112806**. Run: 189. Construction start: 189.

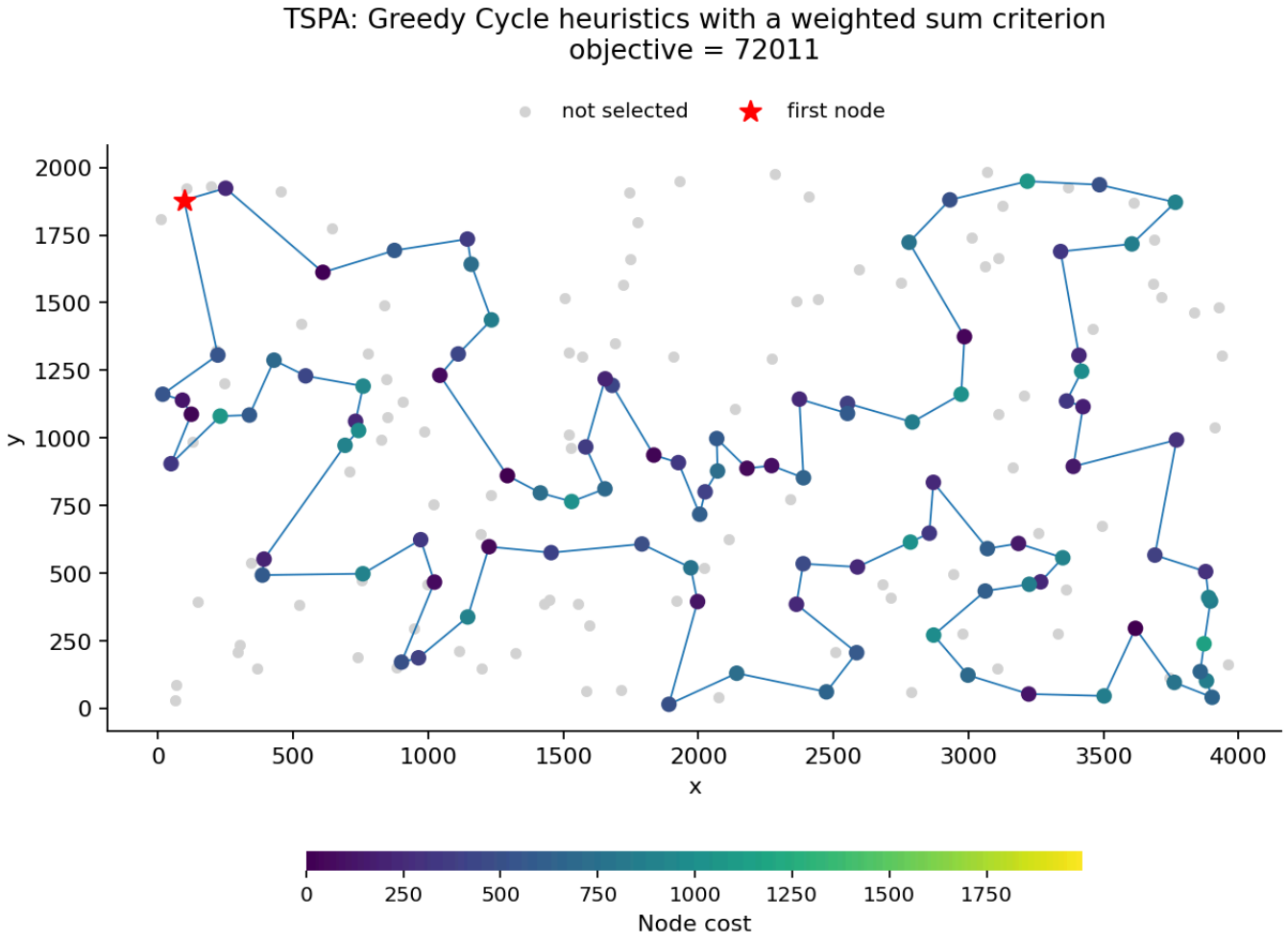


Solution (tour order):

```
189 166 183  10  70  22  25 114 118 181
121  61  41 140  53 179 177  36  99  84
134  72  65  20 192 111 182 165 151 106
 89 124  57  64 107  88  78  26  75 170
159 130 117  93  50  47  37 147   6  46
 86  55  17  29  19 163 197 136 195 184
164 142  85  81 174 194   9  33 143  62
 34 154  39 175 115   5 112 139  18   4
196  51 190  67 185 156  28 180 191 141
 30 160  80  82  11 168 188 120 125 187
```

Greedy Cycle heuristics with a weighted sum criterion

Objective: **72011**. Run: 31. Construction start: 31.

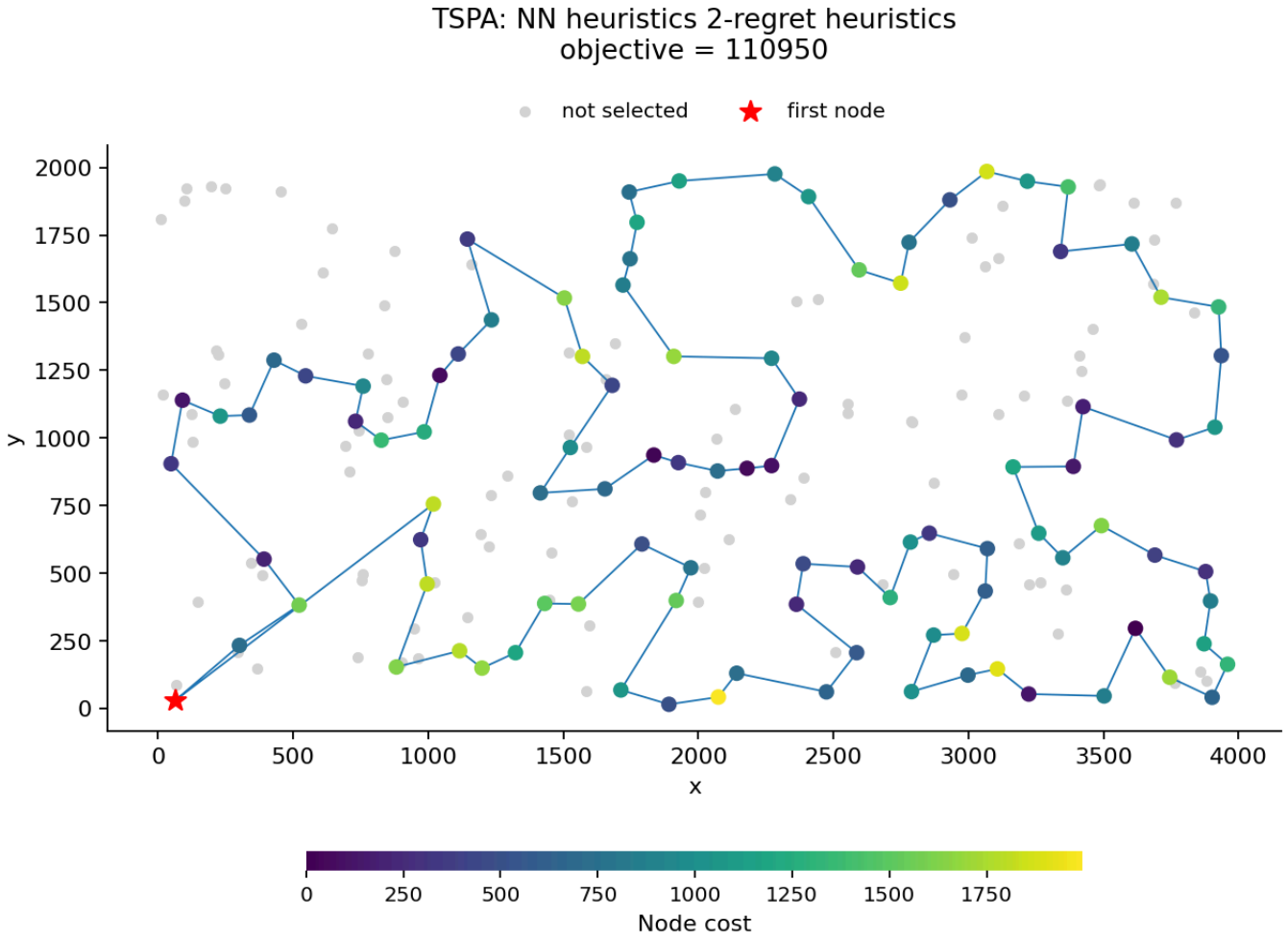


Solution (tour order):

```
 31  60 116 149  63 168  11  49 108 152
100 123  83 160  97 127 161  30 187 171
  2 189 141  87  45  73 135 114 181 192
158  58  84 134  72 176 174 150 106 113
 89  42 151 138 182  57  64 107  88  78
193 153  75 170 146 159 130  47  93  50
  3 169   1 178 101  55  29 163 136 186
164 142   8   9  54  33 131  36 177  52
179  53 140 144  67  16  51  35 185 102
 28 180 132  76   4  71 157  91  80  48
```

NN heuristics 2-regret heuristics

Objective: **110950**. Run: 4. Construction start: 4.

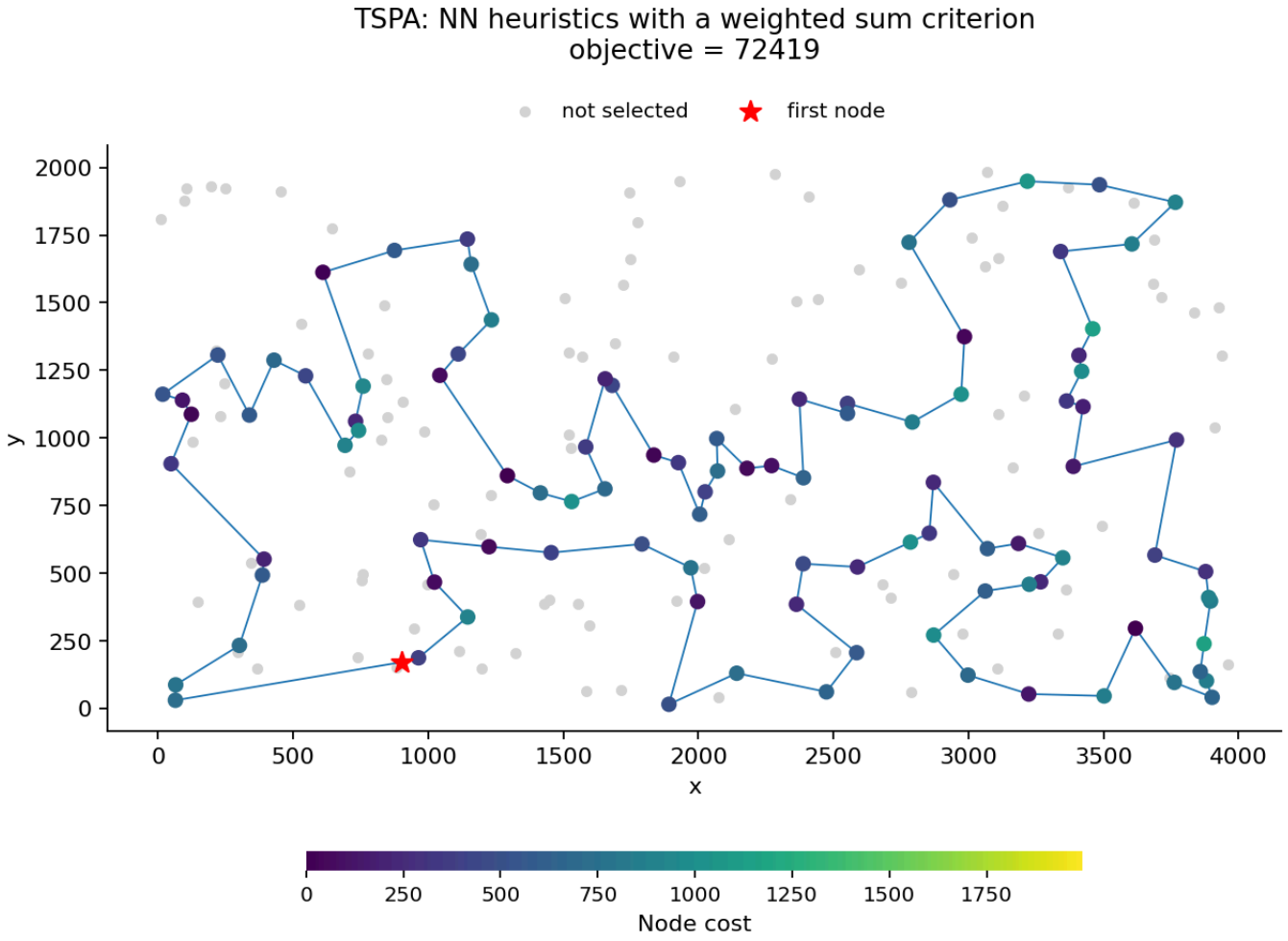


Solution (tour order):

```
104  69 120  97 168 149  11  49 108 152
100 123 148 162 132  76   4 157  18 173
 16 156  28 185  67 144 179 177  36  33
143 190 139 112   5 115 175  39 154  34
109 186 136 197 163  19 178 101 155  21
147  37  47  50  93  81  43 113 117 130
159 170  75  26  78 122 107  64  57 105
182 111 138 165 151 150 176  72  65 134
 84  58 158 192 181 118 114  25 121  73
 45  70  96  10 183 166  32  40  30  23
```

NN heuristics with a weighted sum criterion

Objective: **72419**. Run: 69. Construction start: 69.



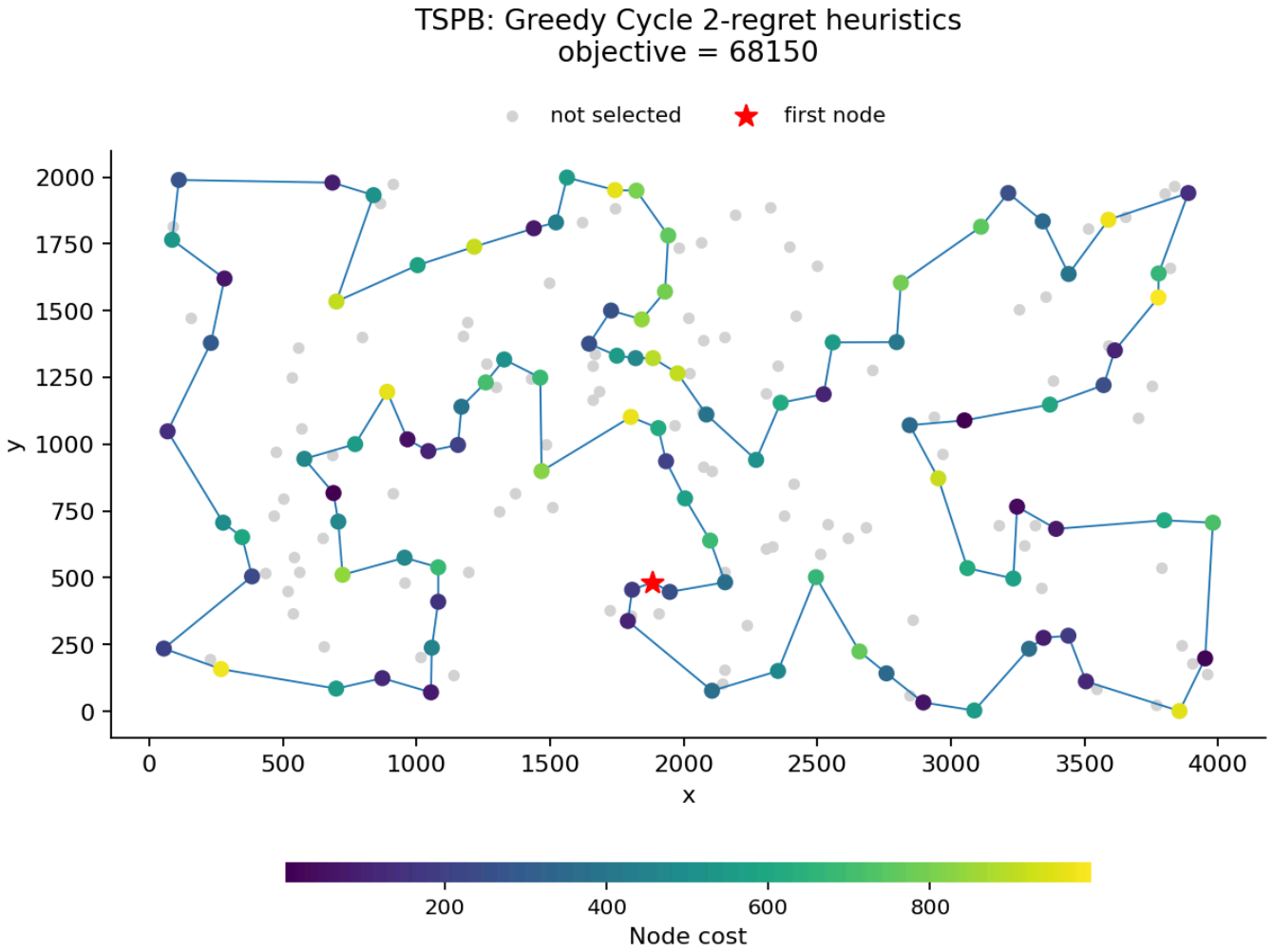
Solution (tour order):

```
171   2 189 187  30 141  87  45  73 135
114 181 192 158  58  84 134  72 176 174
150 106 113  89  42 151 138 182  57  64
107  88  78 193 153  75 170 146 159 130
 47  93  50   3 169   1  59 178 101  55
 29 163 136 186 164 142   8   9  54  33
131  36 177  52 179  53 140 144  67  16
 51  35 185 102  28 180 132  76   4  71
157  91  80 100 123  83 160 152 108  49
 60 116 149  63 168  97 127  69 129 104
```

TSPB

Greedy Cycle 2-regret heuristics

Objective: **68150**. Run: 73. Construction start: 73.

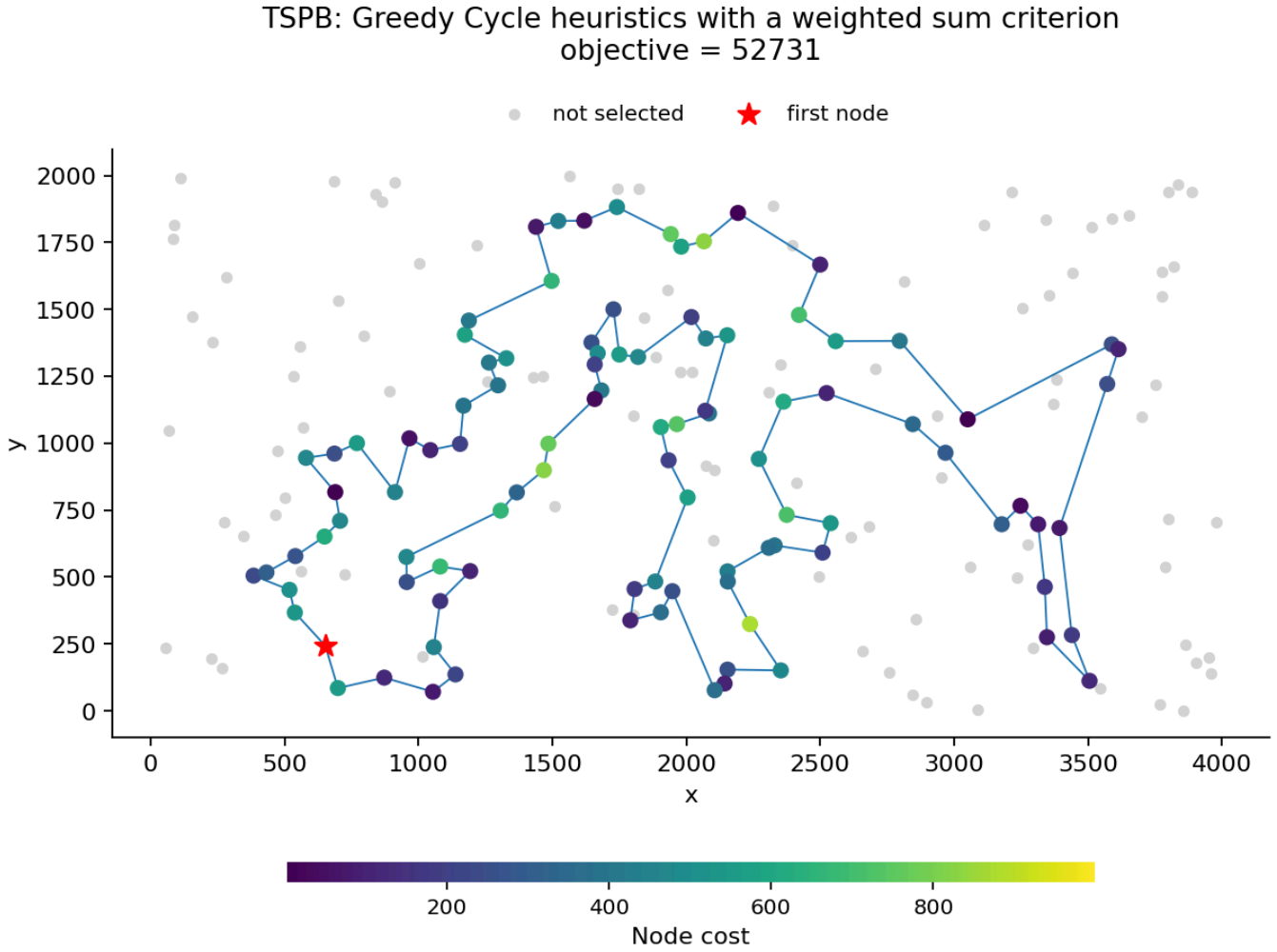


Solution (tour order):

```
 73 167 181  25  98 151 122 128   4 168
183 176  38  47 140 154 161  26 177 170
115 113 155 141  60 157 134 175  62 108
  1 132 103 178  65 145 149  13  37 116
147 123  50 195 121 198  58  16 164  43
166  92 120 133   8 117  95 109   3  27
135  76 185 104  77 194 190  70 187 165
105 139 193 114  10  78  54 162 106 150
182 127  90  56 131 110  23   2  14  20
 74  17 171  99 163  31  88  82  85  59
```

Greedy Cycle heuristics with a weighted sum criterion

Objective: **52731**. Run: 91. Construction start: 91.

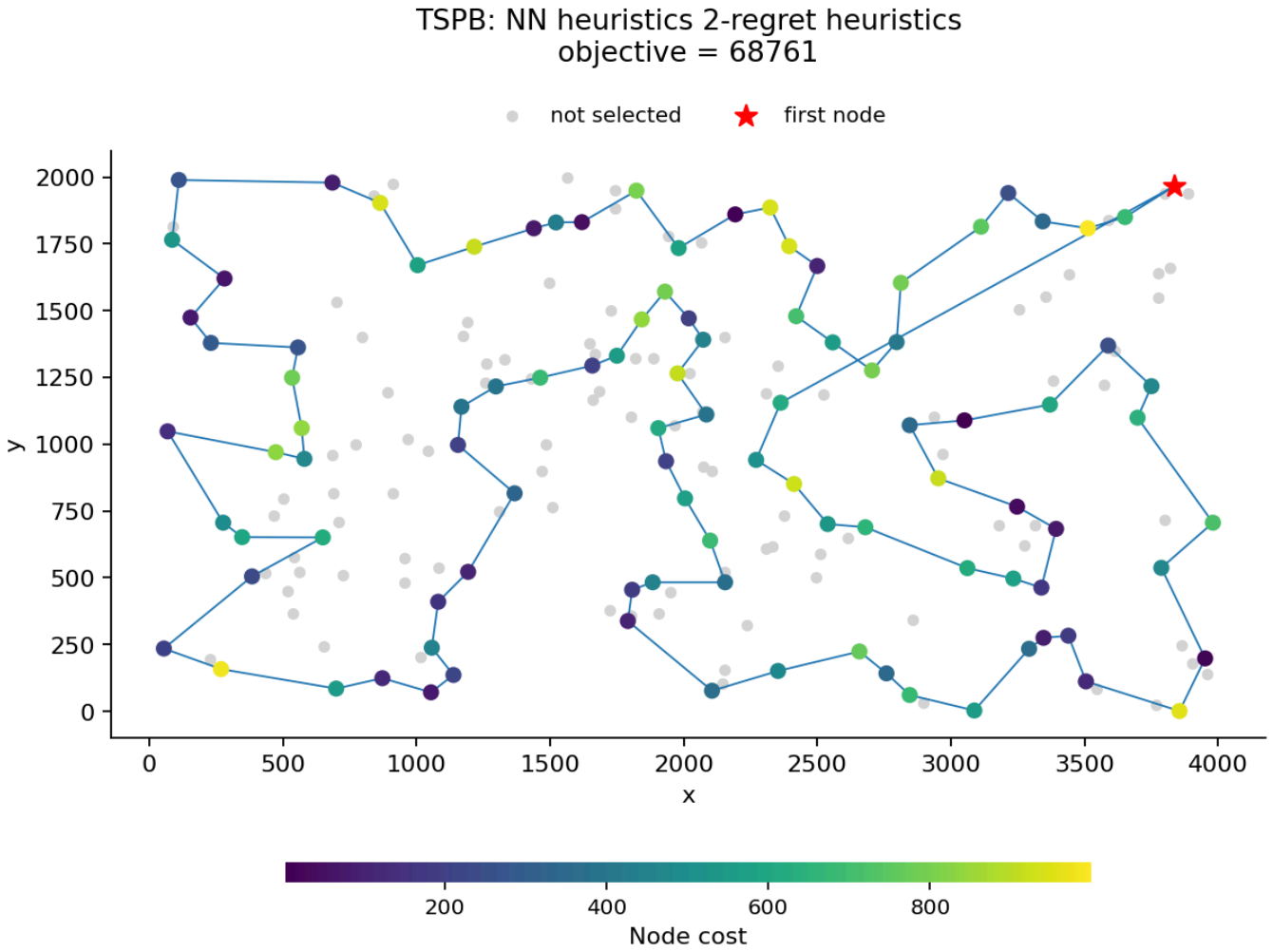


Solution (tour order):

```
 91  36 197 132  29   6 180 115 170 177
125  26  97 154 140  47  38  21  30 183
191 130  63 121 198  96  48  43 156  32
 89  57  79 185 104  54 196 114  10  90
 14   2  20 153 129 127  80 192 162  76
135  27  11  66  67  24  34 152 181 126
 88 179  87  82 167  51  85  59  73  98
151 122 173   3 184  86  46  72 117   8
120 133 136 172 160  69 124   4  64  33
155  71 141  40  60 157  45 134 175  62
```

NN heuristics 2-regret heuristics

Objective: **68761**. Run: 138. Construction start: 138.

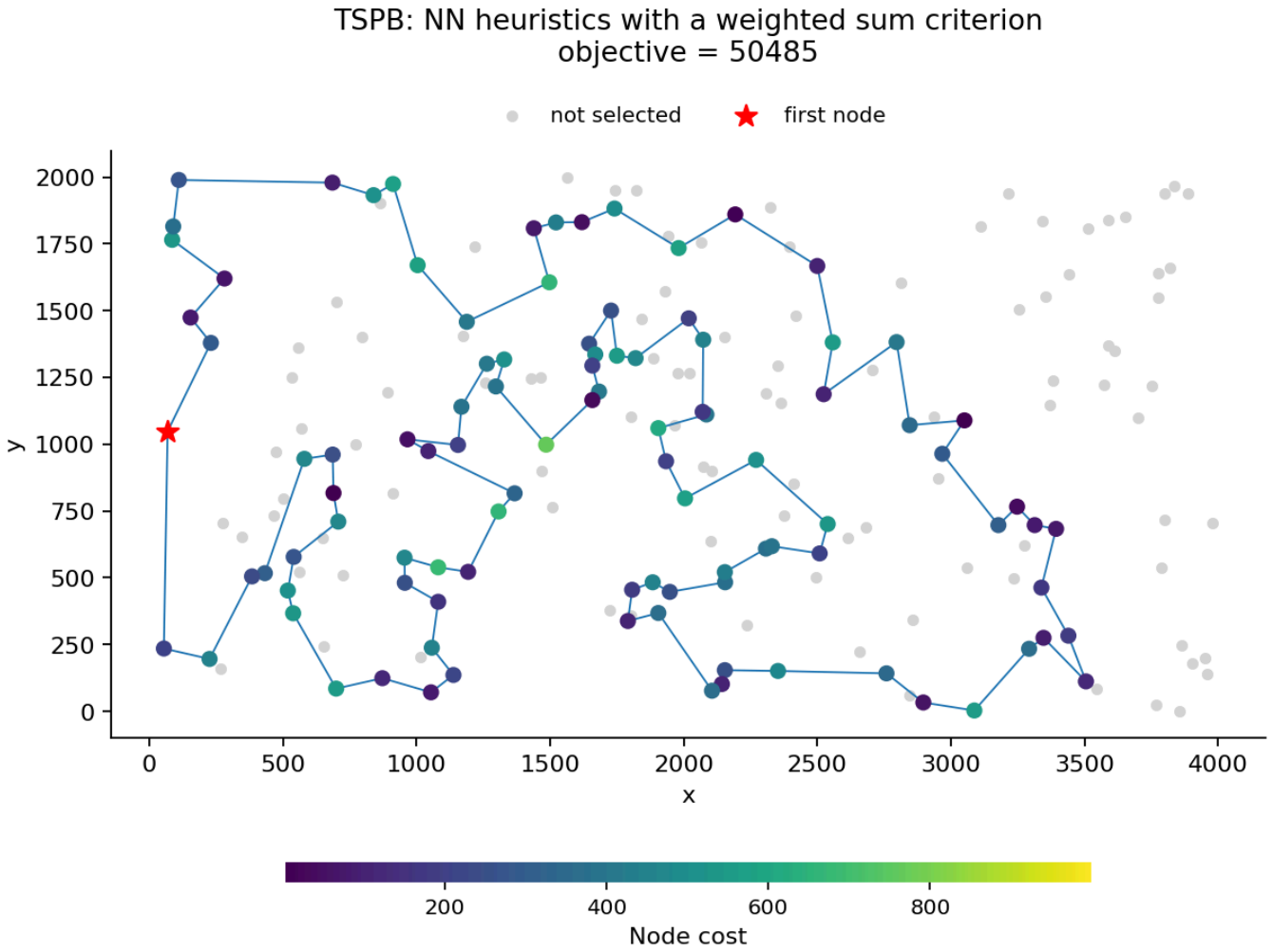


Solution (tour order):

```
 94 169 199  70 190 194  77 104 144 185
 79  57  83  44  89 156 164  96 198 121
195  50   7 116  37  13 149  28 145 158
138  55 177  53  65 178 103 180 132   1
108  62 175 134  45 157  60  40  64  47
 38  21 168 172   8  92 166  72  46 109
  3 122 151  98  25 181  73  59  85  82
 88 163  99  39  17  74  20  14   2  23
110 118 131 143 148 196  78  54 162 106
127  90 153 182 150  84  66  35  27 135
```

NN heuristics with a weighted sum criterion

Objective: **50485**. Run: 192. Construction start: 192.



Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
 50 130  63 121 198  96  48 156  89  57
185  76 104 162  54 192  80 127 129  90
153  14   2  20  74  17 171  99  88 179
 87  82  51  85  59  73 167 181 152  34
 24  67  66  27  98 151 122   3 184  46
 72 117   8 120 133 136 172 160  69 124
 21 183  30  38  47 154 140  64  33  40
141 155  71  60 157  45 134 175  62  36
197   6 115 170 125 177  29 132 174   1
```

Solution checking

All 1600 exported lab 2 solutions were checked for exactly 100 distinct indices in the range 0-199. Their objectives were independently recalculated from the input coordinates using rounded Euclidean distances, including the closing edge and each selected node cost exactly once. All recalculated objectives match the exported values. Minimum, maximum, and mean objectives were also checked against the lab 2 summary. The provided Excel checker was not used for this verification.

Source code

Repository: PiotrFoltyniewicz/Evolutionary-Computation.

- 2-regret and weighted algorithms
- Insertion cost calculation
- Objective evaluation
- Experiment statistics and timing
- Experiment runner
- Visualization script
- Exported solutions
- Lab 2 summary
- Lab 1 summary

Conclusions

Weighted variants outperform pure 2-regret on both instances by balancing regret with insertion cost.

Weighted greedy cycle performs best on TSPA (mean 73353.51, minimum 72011), while weighted nearest neighbor leads on TSPB (mean 55698.34, minimum 50485). Both improve on the best lab 1 results, although weighted nearest neighbor on TSPA is worse on average than its lab 1 counterpart.

These quality improvements come at a higher runtime: approximately 5.08-5.46 ms per run, compared with 2.88-4.49 ms for the corresponding lab 1 insertion methods.